APPENDIX A
ALGORITHM

As shown in Figure 1, we provide a complete pseudocode representation of the EVOREPAIR workflow, which delineates the step-by-step logic and decision-making mechanisms integrated into each remediation cycle.

Algorithm 1: Self-Evolving Framework for Automated Vulnerability Repair

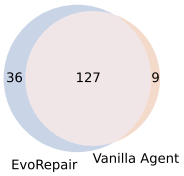
Input: Current vulnerability q, initial experience bank E
Output: Final patched code
begin
  // Initialize variables
  $T \leftarrow 0$; `// Repair turns`
  $V \leftarrow$ Unfixed vulnerabilities
  while V *is not empty* **do**
    $T \leftarrow T + 1$
    // Step 1: Retrieve relevant experiences
    $S \leftarrow$ Experience Retrieval(q, E)
    // Step 2: Prioritize retrieval based on quality and similarity
    $R \leftarrow$ Prioritize(S)
    // Step 3: Inject retrieved experiences into the repair context
    Inject(R)
    // Step 4: Perform repair using the vanilla agent
    Patch $\leftarrow$ VanillaAgentRepair(q)
    // Step 5: Validate patch and update experience bank
    if *Validation(Patch, q)* **then**
      $E \leftarrow$ UpdateExperience$(Patch, T)$
      $V \leftarrow V \backslash \{q\}$; `// Remove fixed vulnerability`
  return Final patched code

APPENDIX B
FULL RESULT ON SEC-BENCH

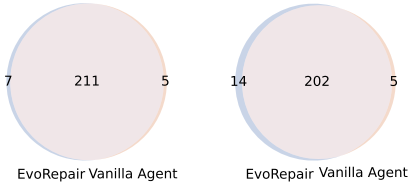
We present in Table I the complete results of 15 turns of repairs on SEC-bench for the Vanilla agent, Live-SWE-Agent, and EVOREPAIR. The results show that EVOREPAIR generally outperforms both Live-SWE-Agent and the Vanilla agent. When using Qwen3.5-Plus as the base model, EVOREPAIR achieves the most pronounced improvement, fixing 26 more vulnerabilities than Live-SWE-Agent and 76 more than the Vanilla agent. With DeepSeek-v3.2 and GPT-5-mini, although EVOREPAIR's final repair ceiling is roughly on par with Live-SWE-Agent, EVOREPAIR typically converges 3–5 turns faster. Additionally, in Tables II and III we report each method's file-level and function-level localization accuracy at every turn.



(a) GPT-5-mini (b) Qwen3.5-Plus



(c) DeepSeek-v3.1



(d) Devstral-24B (e) Devstral-123B

APPENDIX C
OVERLAP ANALYSIS

Overlap analysis between vanilla agent and EVOREPAIR. on PATCHEVAL, we conduct an overlap analysis of the vulnerabilities remediated by EVOREPAIR and the vanilla agent to evaluate their respective repair capabilities. As illustrated in Figures 1(a) to 1(e), we compare the CVEs remediated by EVOREPAIR and the vanilla agent across various base models. We observe a high degree of overlap between the vulnerabilities resolved by the two methods. Notably, EVOREPAIR effectively encompasses the remediation coverage of the vanilla agent; specifically, when using Qwen3.5-Plus as the base model, there is only one vulnerability that EVOREPAIR fails to remediate while the vanilla agent succeeds. This indicates that EVOREPAIR successfully preserves the baseline agent's strengths while simultaneously augmenting its repair capabilities. These results demonstrate the robustness of EVOREPAIR, confirming that the integration of the experience library does not introduce excessive noise or degrade the original repair performance of the base model.

Overlap analysis between different models. Beyond analyzing the overlap in remediated vulnerabilities across different methods, we also investigate the overlap among various base models. As illustrated in Figure 1(f), we observe that even prior to the integration of EVOREPAIR, the vanilla agents across different base models already exhibit a significant degree of overlap in their remediation coverage. Specifically, the 129 jointly repaired vulnerabilities account for 60% or more of the successful repairs for any individual model, indicating

TABLE I: Comparison of test-time scaling performance (EVOREPAIR, Live-SWE-Agent, Vanilla Agent) on SEC-bench.

Model	Method	Turn 1	Turn 2	Turn 3	Turn 4	Turn 5	Turn 6	Turn 7	Turn 8	Turn 9	Turn 10	Turn 11	Turn 12	Turn 13	Turn 14	Turn 15	# Fix
DeepSeek-v3.1	Vanilla	22	50	65	75	83	90	95	98	99	101	111	114	117	118	118	118
	Live-SWE-Agent	28	59	73	83	95	102	108	112	120	121	125	127	127	-	-	127
	EVOREPAIR	44	73	90	102	110	112	116	120	125	126	131	131	-	-	-	131
DeepSeek-v3.2	Vanilla	94	122	140	149	149	-	-	-	-	-	-	-	-	-	-	149
	Live-SWE-Agent	86	122	135	146	150	154	159	163	166	167	171	173	173	-	-	173
	EVOREPAIR	101	134	147	156	163	168	169	171	173	173	-	-	-	-	-	173
Qwen3.5-Plus	Vanilla	36	60	71	77	78	82	87	89	94	94	-	-	-	-	-	94
	Live-SWE-Agent	67	84	101	111	116	121	125	126	128	131	134	137	138	141	144	144
	EVOREPAIR	64	98	120	135	141	146	149	152	156	159	165	168	170	170	-	170
GPT-5-mini	Vanilla	51	81	98	110	122	128	131	138	142	145	150	152	153	155	155	155
	Live-SWE-Agent	35	74	102	117	125	132	141	145	148	151	152	156	157	160	160	160
	EVOREPAIR	66	119	135	142	149	151	155	158	161	164	166	170	171	174	174	174

TABLE II: File-level localization accuracy on SEC-bench.

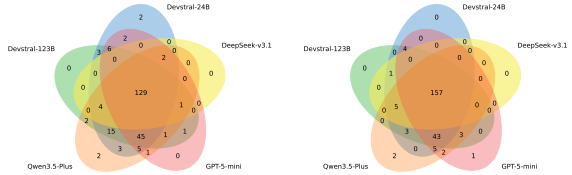
Model	Method	Turn 1	Turn 2	Turn 3	Turn 4	Turn 5	Turn 6	Turn 7	Turn 8	Turn 9	Turn 10	Turn 11	Turn 12	Turn 13	Turn 14	Turn 15
DeepSeek-v3.1	Vanilla	49.07%	41.76%	43.55%	38.60%	41.46%	40.82%	45.45%	35.42%	30.43%	51.52%	52.17%	25.45%	54.17%	50.00%	32.14%
	Live-SWE-Agent	41.28%	39.60%	33.77%	42.86%	27.50%	39.47%	48.15%	25.93%	33.96%	44.00%	35.71%	25.00%	35.14%	-	-
	EVOREPAIR	49.69%	47.75%	41.84%	45.45%	37.65%	39.39%	41.43%	48.28%	42.86%	36.54%	40.35%	36.67%	-	-	-
DeepSeek-v3.2	Vanilla	46.67%	52.88%	56.06%	48.00%	46.81%	-	-	-	-	-	-	-	-	-	-
	Live-SWE-Agent	47.08%	52.73%	48.15%	52.24%	50.00%	47.37%	50.00%	41.18%	59.26%	41.18%	44.83%	57.69%	52.63%	-	-
	EVOREPAIR	47.26%	45.76%	35.29%	38.71%	40.74%	42.50%	63.33%	46.43%	50.00%	47.83%	-	-	-	-	-
Qwen3.5-Plus	Vanilla	41.18%	61.90%	42.42%	41.38%	38.46%	47.37%	57.89%	28.57%	54.17%	63.64%	-	-	-	-	-
	Live-SWE-Agent	62.16%	38.33%	53.33%	57.14%	57.89%	47.37%	66.67%	46.15%	62.50%	50.00%	63.64%	53.85%	54.55%	87.50%	60.00%
	EVOREPAIR	53.23%	54.79%	55.81%	50.00%	65.22%	61.11%	27.78%	33.33%	72.73%	53.33%	53.33%	37.50%	60.00%	40.00%	-
GPT-5-mini	Vanilla	30.94%	32.37%	26.90%	25.62%	27.64%	23.64%	25.00%	30.85%	22.58%	20.43%	25.88%	22.64%	25.42%	36.73%	21.57%
	Live-SWE-Agent	33.07%	30.97%	27.63%	27.42%	19.81%	23.85%	21.21%	23.26%	16.25%	36.36%	18.75%	20.00%	18.97%	15.09%	20.00%
	EVOREPAIR	33.20%	42.13%	37.89%	35.00%	42.86%	33.80%	32.81%	40.00%	33.33%	38.30%	34.88%	32.61%	45.71%	43.59%	42.11%

TABLE III: Method-level localization accuracy on SEC-bench.

Model	Method	Turn 1	Turn 2	Turn 3	Turn 4	Turn 5	Turn 6	Turn 7	Turn 8	Turn 9	Turn 10	Turn 11	Turn 12	Turn 13	Turn 14	Turn 15
DeepSeek-v3.1	Vanilla	38.20%	34.62%	26.92%	21.43%	24.32%	20.45%	22.45%	21.43%	20.51%	43.75%	34.09%	20.45%	29.63%	21.88%	13.79%
	Live-SWE-Agent	31.63%	29.03%	35.29%	31.48%	18.52%	41.94%	27.59%	28.21%	25.00%	26.92%	24.39%	17.86%	33.33%	-	-
	EVOREPAIR	36.69%	29.25%	26.74%	28.92%	29.63%	25.76%	25.68%	28.57%	30.00%	23.53%	30.00%	20.37%	-	-	-
DeepSeek-v3.2	Vanilla	40.20%	34.26%	39.44%	29.41%	31.91%	-	-	-	-	-	-	-	-	-	-
	Live-SWE-Agent	39.38%	33.88%	40.28%	38.81%	28.30%	28.57%	29.79%	30.77%	33.33%	22.22%	21.21%	23.33%	26.32%	-	-
	EVOREPAIR	41.84%	36.52%	30.99%	38.71%	27.12%	31.71%	47.22%	21.21%	31.25%	23.08%	-	-	-	-	-
Qwen3.5-Plus	Vanilla	41.94%	50.00%	42.31%	25.00%	33.33%	19.05%	38.10%	16.67%	40.00%	40.00%	-	-	-	-	-
	Live-SWE-Agent	52.08%	36.17%	50.00%	42.11%	40.00%	31.58%	53.85%	35.71%	27.27%	20.00%	42.86%	37.50%	38.46%	62.50%	37.50%
	EVOREPAIR	47.47%	48.28%	35.71%	36.67%	46.43%	34.78%	20.00%	13.33%	33.33%	25.00%	23.53%	20.00%	40.00%	25.00%	-
GPT-5-mini	Vanilla	22.62%	23.18%	14.18%	18.81%	17.31%	15.38%	13.95%	13.25%	12.99%	8.86%	12.99%	8.93%	14.04%	16.67%	12.00%
	Live-SWE-Agent	28.97%	27.42%	22.63%	20.39%	12.24%	17.20%	17.07%	16.44%	13.04%	20.45%	6.67%	11.48%	9.80%	7.32%	11.36%
	EVOREPAIR	23.41%	33.55%	24.18%	34.21%	33.33%	20.63%	24.07%	25.00%	21.15%	16.67%	11.36%	19.05%	24.24%	24.32%	21.62%

a substantial level of inherent consistency in how different LLMs handle standard security flaws.

However, upon the introduction of EVOREPAIR, as shown in Figure 1(g), the number of shared successful repairs across the five models increases to a striking 157, representing over 70% of any single model's repair set. This significant expansion in the overlap set demonstrates that EVOREPAIR successfully preserves the original strengths of the vanilla agents while consistently augmenting their capabilities. More importantly, this trend suggests that the experience library acts as a powerful 'knowledge anchor,' harmonizing the repair trajectories of diverse base models toward a convergent, optimal solution space. It further proves that EVOREPAIR possesses excellent robustness, as the high-quality diagnostic priors effectively guide different models to resolve complex vulnerabilities without introducing excessive noise that might otherwise degrade their baseline performance.



(f) Vanilla Agent (g) EVOREPAIR

APPENDIX D
COMBINED WITH LIVE-SWE-AGENT

During the reproduction of Live-SWE-Agent, we observe that while synthesizing bash tools does not achieve full self-evolution, with multi-turn convergence results falling between the vanilla agent and EVOREPAIR, combining fine-grained bash commands into fixed-function scripts effectively reduces the length of the agent's repair trajectories and accelerates convergence.

Driven by these observations, we conduct further exploratory experiments by integrating Live-SWE-Agent with EVOREPAIR, aiming for a breakthrough in both performance and efficiency.

Specifically, we incorporate the prompt (as shown below) from Live-SWE-Agent, which instructs the model to synthesize new tools during the repair process, into the EVOREPAIR prompt while maintaining all other configurations. We evaluate this hybrid approach across three models (Qwen3.5-Plus, Devstral-24B, and DeepSeek-v3.1) on PATCHEVAL. The experimental results indicate the following:

(1) Performance ceiling. The integration of Live-SWE-

Agent does not significantly raise the upper bound of repair capabilities. Performance remains on par with the original results for Devstral-24B and DeepSeek-v3.1, while Qwen3.5-Plus successfully fixes three additional vulnerabilities.

(2) Model efficiency. The integration shows a measurable impact on efficiency. Both Qwen3.5-Plus and Devstral-24B reach convergence five turns earlier than their respective baselines.

Combined with the experimental results above, we believe that tool evolving and experience evolving are two critical pathways to achieving self-evolving agents. Among them, experience evolving focuses more on raising the performance ceiling, while tool evolving focuses more on enhancing efficiency. The organic integration of both can achieve a superior outcome that is both fast and effective.

Tool evolving prompt in Live-SWE-Agent

Creating your own tools
- You can also create your own tools in Python to help with your workflow
- Compared to basic bash commands, the tools you create should be able to better aid your workflow in solving the task
- Ensure each tool you create is in Python, contains informative outputs or error messages, and can be ran from the command line
- You should at least create a simple edit tool that can help you effectively edit arbitrary files instead of using bash commands
- The tools you create can be for any purpose, it does not need to be general, instead think about how it can help you specifically with the current task at hand
Example of creating a custom tool:
<example_response>
THOUGHT: I noticed that in order to solve the issue I need to ... therefore I should create a custom tool to help me ...
```bash
cat <<'EOF' > /path/to/tool_name.py
#!/usr/bin/env python3
import sys
# Import other packages if needed
def main():
    # Your tool logic here
    ...
if __name__ == "__main__":
    main()
EOF
```
</example_response>
Example of using the tool you created:
<example_response>
THOUGHT: Let me use the custom tool I created to help me with ...
```bash
python /path/to/tool_name.py <<EOF
your_input_here
EOF
```

APPENDIX E
EXPERIENCE TRANSFER

As shown in Table IV, we present the full results of the experience transfer experiment. Specifically, we conduct experiments under three configurations, where the teacher model is consistently Qwen3.5-Plus, and the student models are Qwen3.5-Plus, Qwen3-Max, and Qwen3-Coder, respectively. We transfer the experience generated by EVOREPAIR using Qwen3.5-Plus on PATCHEVAL to VUL4J, and then compare it with the repair results without experience transfer under the same model. Overall, experience transfer demonstrates effectiveness across all models. Specifically, Qwen3.5-Plus fixes 3 additional vulnerabilities (31 $\rightarrow$ 34), Qwen3-Max fixes 2 additional vulnerabilities (23 $\rightarrow$ 25), and Qwen3-Coder-Plus fixes 1 additional vulnerability (22 $\rightarrow$ 23). By examining the repair details of each vulnerability in Table IV, we observe that the model's repair behavior changes after applying experience transfer. Specifically, vulnerabilities that the model consistently fails to repair without prior knowledge can now be successfully fixed, as the introduced experience provides valuable guidance for the model to discover correct patches. Taking Qwen3.5-Plus as an example, the three vulnerabilities VUL4J-56, VUL4J-58, and VUL4J-71 cannot be fixed consistently by the model across multiple independent runs without experience guidance. However, after introducing experience transfer, the model achieves stable and reliable repair for these vulnerabilities. On the other hand, the introduced experience may also bring noise. For instance, Qwen3-Max can successfully repair VUL4J-25 without experience guidance but fails after applying experience transfer. This phenomenon is more pronounced in cross-model transfer (e.g., Qwen3.5-Plus $\rightarrow$ Qwen3-Max, Qwen3.5-Plus $\rightarrow$ Qwen3-Coder-Plus), while the effectiveness of experience is relatively robust under identical-model transfer (e.g., Qwen3.5-Plus $\rightarrow$ Qwen3-Plus). Besides the model choice, experience transfer is also affected by many other factors. For instance, the vulnerabilities in PATCHEVAL are written in JavaScript, Go, and Python, whereas VUL4J contains Java programs. The differences among programming languages impose a non-negligible impact on the effectiveness of experience transfer. As a result, the performance of experience transfer across datasets, languages, and models degrades compared with experience evolution under the single dataset, same model, and identical language setting. In future work, we will further investigate how to improve the effectiveness of experience transfer. We plan to enhance its performance by extracting and transforming knowledge across programming languages, as well as by reducing the amount of experience to alleviate noise interference.

APPENDIX F
CASE STUDY

To better understand the efficacy of our experience-based approach, we conduct a systematic case study on vulnerabilities that EVOREPAIR successfully remediates but which remain intractable for vanilla agents. By examining diverse repair trajectories, ranging from successful strategies to lessons learned from past failures, we isolate the critical mechanisms through which our experience library enhances agent robustness and problem-solving efficiency.

EVOREPAIR leverages past failures as valuable lessons to avoid repeating mistakes in current repair trajectories. Taking the remediation of CVE-2020-8132 by GPT-5-mini as an example, by internalizing the failure experiences gained from the earlier repair of CVE-2020-7795, the model successfully resolves the vulnerability. In contrast, without such experience-based guidance, GPT-5-mini fails to remediate CVE-2020-8132. As illustrated in the figure below, we list the specific experiences utilized by EVOREPAIR when remediating CVE-2020-8132.

Furthermore, we observe how EVOREPAIR applies these positive experiences to novel challenges, such as the remediation of CVE-2022-0691 in the `url-parse` package. While the vanilla agent misdiagnoses the flaw as prototype pollution and attempts to over-engineer a complex wrapper, EVOREPAIR identifies the root cause (an incomplete whitespace regex) by referencing prior experience in robust input normalization. Guided by the experience rule, *"Avoid regex/text manipulation for structured code edits; prefer AST-based or line-precise tools,"* EVOREPAIR replaces fragile `sed` scripts with precise Python-based line modification. This transition from brittle text processing to structurally sound edits ensures that the fix passes all test cases without inducing syntax errors.

The success of EVOREPAIR in these cases is attributed to three critical cognitive mechanisms:

a. Shift from Blind Mutation to Structured Diagnostic Reasoning. While the vanilla agent reacts to patch application failures by blindly attempting complex, fragile regex transformations, ultimately inducing syntax errors, EVOREPAIR adopts a disciplined "suspend-diagnose-replan" workflow. By internalizing the rule to "test patch content before submission" and "verify workspace state," the agent effectively breaks the iterative error loop, transforming a failed deployment into a solvable debugging task.

b. Enforcement of High-Confidence Security Paradigms. The library functions as a cognitive anchor against "repair depth degradation." When the vanilla agent faces prolonged difficulty, it regresses to superficial, unsafe fixes (e.g., simple regex-based whitelisting). In contrast, EVOREPAIR relies on explicit library guidance, such as *"Prefer execFile over exec"*, to maintain a consistent security baseline, ensuring root-cause remediation of CWE-94.

c. Mitigation of Environmental and Toolchain Entropy. A significant failure mode of the vanilla agent is "patch pollution," where irrelevant modifications (e.g., to `package-lock.json`) render patches non-applicable. EVOREPAIR mitigates this by strictly adhering to targeted patch-generation rules, such as `git diff -- . ':(exclude)*test*'`. By ensuring minimal, precise code edits, the agent preserves repository integrity, allowing automated verification to succeed. Both experiences used by EVOREPAIR are listed as follows.

TABLE IV: Experience transfer result on the VUL4J dataset.

VUL	Qwen3.5-Plus Vanilla	Qwen3.5-Plus Transfer	Qwen3-Max Vanilla	Qwen3-Max Transfer	Qwen3-Coder-Plus Vanilla	Qwen3-Coder-Plus Transfer
VUL4J-1	✗	✗	✗	✗	✗	✗
VUL4J-2	✓	✓	✓	✓	✓	✓
VUL4J-3	✗	✗	✗	✗	✗	✗
VUL4J-4	✗	✗	✗	✗	✗	✗
VUL4J-5	✗	✗	✗	✗	✗	✗
VUL4J-6	✗	✗	✗	✗	✗	✗
VUL4J-7	✗	✗	✗	✗	✗	✗
VUL4J-8	✗	✗	✗	✗	✗	✗
VUL4J-9	✓	✓	✓	✓	✓	✓
VUL4J-10	✓	✓	✗	✓	✗	✓
VUL4J-11	✓	✓	✓	✓	✓	✓
VUL4J-12	✗	✗	✗	✗	✗	✗
VUL4J-13	✗	✗	✗	✗	✗	✗
VUL4J-14	✗	✗	✗	✗	✗	✗
VUL4J-15	✗	✗	✗	✗	✗	✗
VUL4J-16	✗	✗	✗	✗	✗	✗
VUL4J-17	✗	✗	✗	✗	✗	✗
VUL4J-18	✓	✓	✓	✓	✗	✓
VUL4J-19	✓	✓	✗	✗	✗	✓
VUL4J-20	✓	✓	✗	✓	✓	✗
VUL4J-21	✗	✗	✗	✗	✗	✗
VUL4J-22	✓	✓	✗	✗	✓	✗
VUL4J-23	✗	✗	✗	✗	✗	✗
VUL4J-24	✗	✗	✗	✗	✗	✗
VUL4J-25	✓	✓	✓	✗	✗	✗
VUL4J-26	✗	✗	✗	✗	✗	✗
VUL4J-27	✗	✗	✗	✗	✗	✗
VUL4J-28	✗	✗	✗	✗	✗	✗
VUL4J-29	✗	✗	✗	✗	✗	✗
VUL4J-30	✗	✗	✗	✗	✗	✗
VUL4J-31	✗	✗	✗	✗	✗	✗
VUL4J-32	✗	✗	✗	✗	✗	✗
VUL4J-33	✗	✗	✗	✗	✗	✗
VUL4J-34	✗	✗	✗	✗	✗	✗
VUL4J-35	✗	✗	✗	✗	✗	✗
VUL4J-36	✓	✓	✗	✗	✗	✓
VUL4J-37	✗	✗	✗	✗	✗	✗
VUL4J-38	✓	✓	✓	✓	✗	✗
VUL4J-39	✗	✗	✗	✗	✗	✗
VUL4J-40	✗	✗	✗	✗	✗	✗
VUL4J-41	✓	✓	✓	✓	✗	✗
VUL4J-42	✗	✗	✗	✗	✗	✗
VUL4J-43	✓	✓	✓	✓	✓	✓
VUL4J-44	✗	✗	✗	✗	✗	✗
VUL4J-45	✓	✓	✓	✓	✓	✓
VUL4J-46	✗	✗	✗	✗	✗	✗
VUL4J-47	✗	✗	✗	✗	✗	✗
VUL4J-48	✓	✓	✓	✓	✓	✓
VUL4J-49	✓	✓	✓	✓	✓	✓
VUL4J-50	✗	✗	✗	✗	✗	✗
VUL4J-51	✗	✗	✗	✗	✗	✗
VUL4J-52	✓	✓	✓	✗	✗	✓
VUL4J-53	✗	✗	✗	✗	✗	✗
VUL4J-54	✗	✗	✗	✗	✗	✗
VUL4J-55	✗	✗	✗	✗	✗	✗
VUL4J-56	✗	✓	✗	✗	✓	✓
VUL4J-57	✓	✓	✗	✓	✓	✗
VUL4J-58	✗	✓	✗	✗	✗	✗
VUL4J-59	✓	✓	✓	✓	✗	✗
VUL4J-60	✓	✓	✗	✗	✓	✓
VUL4J-61	✓	✓	✓	✓	✓	✓
VUL4J-62	✓	✓	✓	✓	✓	✓
VUL4J-63	✗	✗	✗	✗	✗	✗
VUL4J-64	✓	✓	✓	✓	✓	✓
VUL4J-65	✓	✓	✓	✓	✓	✓
VUL4J-66	✓	✓	✓	✓	✗	✗
VUL4J-67	✗	✗	✗	✗	✗	✗
VUL4J-68	✓	✓	✓	✓	✓	✓
VUL4J-69	✓	✓	✓	✓	✓	✓
VUL4J-70	✓	✓	✓	✓	✓	✗
VUL4J-71	✗	✓	✓	✓	✓	✓
VUL4J-72	✗	✗	✗	✗	✗	✗
VUL4J-73	✗	✗	✗	✗	✗	✗
VUL4J-74	✗	✗	✗	✗	✗	✗
VUL4J-75	✗	✗	✗	✗	✗	✗
VUL4J-76	✓	✓	✗	✗	✗	✗
VUL4J-77	✓	✓	✓	✓	✓	✓
VUL4J-78	✗	✗	✗	✗	✗	✗
VUL4J-79	✓	✓	✗	✓	✓	✓

Failure experience from CVE-2020-7795 used by CVE-2020-8132

Experience from CVE-2020-7795
CWE Types: CWE-94, CWE-77, CWE-78
Quality Score: 4/5
Similarity: 0.84
Repair Result: failure

Vulnerability Fix Report: CVE-2020-7795 (Command Injection in get-npm-package-version)

A. Vulnerability Introduction and Analysis
Type: Command Injection (CWE-77, CWE-78, CWE-94)
Location: `index.js`, lines 1–28
Root Cause: The vulnerable code uses `child_process.execSync()` with string interpolation to construct OS commands. This allows an attacker to inject arbitrary shell commands via malicious `packageName` or `registry` values (e.g., `pkg; rm -rf /`). Since `execSync` passes the command string to a shell, special characters like `;`, `&`, `|`, or `$()` are interpreted, leading to arbitrary code execution.
Impact: Remote code execution in any application that uses this package with untrusted input.

B. Repair Strategy
The correct mitigation strategy is to **avoid shell interpretation entirely** by:

1) Using `child_process.execFileSync()` instead of `execSync()`.
2) Passing command arguments as an array (e.g., `['npm', 'view', packageName, 'version', ...]`) so inputs are treated as literal arguments, not shell tokens.
3) Adding basic input validation to reject obviously malicious patterns (defense-in-depth).

This approach eliminates the injection surface because the OS executes `npm` directly with arguments, bypassing the shell.

C. Trajectory Analysis
Positive Analysis:

- **Correct identification of root cause**: The assistant correctly recognized that `execSync` with string interpolation was the vulnerability source.
- **Adoption of safe alternative**: Switching to `execFileSync` with argument arrays is the standard, secure pattern for spawning subprocesses in Node.js.
- **Input validation added**: While not strictly necessary if shell is avoided, validating `packageName` and `registry` against allowlists (e.g., $/\hat{}[\backslash w@\backslash/\backslash -.] + [\$]/$) provides defense-in-depth.

Negative Analysis:

- **Patch generation errors**: The assistant repeatedly included unintended changes (e.g., to `package-lock.json`) in `fix.patch`, causing `fix-run.sh` to fail during patch application. This indicates a lack of precision in generating diffs.
- **Incorrect file path handling**: In some attempts, the assistant wrote to `index.js` in the wrong directory or failed to isolate changes to only the vulnerable file.
- **Overcomplicated shell commands**: Using complex here-docs with quoting issues led to syntax errors (Turn 6), delaying progress.

D. Experience Summary
Rule 1: Prefer `execFile/spawn` over `exec` for external commands
Applicability: Node.js, CWE-77/78; Goal: Prevent shell injection; Guidance: Never use `exec()` or `execSync()` with user input. Always use `execFile()` or `spawn()` with argument arrays. UNSAFE: execSync(‘npm view ${$pkg$} version‘); SAFE: execFileSync(’npm’, [’view’, pkg, ’version’]);

Rule 2: Generate minimal, targeted patches
Applicability: General, all languages; Goal: Ensure patch applies cleanly during validation; Guidance: Use `git diff -- . ':(exclude)*test*'` to exclude unrelated files. Verify `fix.patch` contains only intended changes before submission.

Rule 3: Validate input even when using safe APIs

Applicability: Defense-in-depth, Node.js; Goal: Reduce attack surface; Guidance: Apply allowlist regexes to inputs like package names (e.g., /^[\w@\/\-.]+$/). Reject empty or non-string inputs early.

E. Reflection and Improvement

Although the assistant correctly identified the fix strategy (use `execFileSync` + validation), the repair **failed** due to operational errors in patch generation and environment handling. Future efforts should:

- Test patch content before submission (`cat fix.patch` to inspect).
- Ensure only the target file is modified.
- Avoid modifying `package-lock.json` unless absolutely necessary.
- Use `git status` and `git diff` to verify workspace state.

Peer review of patches and incremental testing would improve success rates. The core remediation logic was sound—only the delivery mechanism failed.

Successful experience from CVE-2022-0512 used by CVE-2022-0691

Experience from CVE-2022-0512
CWE Types: CWE-862, CWE-639
Quality Score: 4/5
Similarity: 1.00
Repair Result: success

Vulnerability Fix Report: CVE-2022-0512 in url-parse

A. Vulnerability Introduction and Analysis
CVE ID: CVE-2022-0512
CWE Types: CWE-862 (Missing Authorization), CWE-639 (Authorization Bypass Through User-Controlled Key)
Language: JavaScript (Node.js / NPM package `url-parse`)
Location: `url-parse/index.js`, lines 236–389 and 402–488
Description: The vulnerability allows an attacker to bypass authorization by manipulating the `auth` component of a URL (e.g., `user@:pass@`). When parsed, special characters like `@` or `:` in username/password fields were not properly percent-encoded, leading to incorrect parsing.
Root Cause: In both the constructor (`Url`) and the `set('auth', value)` method:

- The `auth` string was split naively using `.split(':')`, which fails when passwords contain colons.
- Username and password values were assigned **without percent-encoding**, allowing raw `@` or `:` to interfere with URL structure.
- No normalization was applied to handle already-encoded inputs, risking double-encoding or inconsistent behavior.

Impact: An attacker could craft a URL like `http://attacker@:victim-password/` that, when parsed, incorrectly assigns `attacker@` as the username and `victim-password` as the password.

B. Repair Strategy
The fix ensures that:

1) **Auth is split only on the first colon**: Correctly separates username from password.
2) **Username and password are safely normalized**: Existing percent-encodings are decoded first (with error tolerance), then re-encoded using `encodeURIComponent`.

This strategy preserves backward compatibility while eliminating the injection surface, addressing CWE-639 by ensuring user-controlled keys cannot inject structural characters.

C. Trajectory Analysis
Positive Analysis:

- **Test-driven validation**: The assistant repeatedly ran `fix-run.sh` to ensure behavioral correctness.
- **Safe encoding logic**: Introduced a `safeDecode` helper with `try/catch` to avoid crashes.
- **First-colon split**: Replaced `.split(':')` with `indexOf(':')` + `slice()`.

Negative Analysis:

- **Initial over-encoding**: Early attempts applied `encodeURIComponent` without decoding first, causing double-encoding.
- **Fragile patching**: Used complex `perl -pe` and regex-based replacements that broke due to quoting issues.

- **Assumed file paths**: Repeatedly tried to access files from incorrect directories.

D. Experience Summary

Rule 1: Normalize User Input via Decode-Then-Encode

Guidance: When accepting possibly pre-encoded strings, **always decode first (safely), then re-encode**.

```
safeNormalize(s) = try {
   encodeURIComponent(decodeURIComponent(s))
} catch {
   encodeURIComponent(s)
}
```

Rule 2: Split on First Occurrence for Delimited Fields

Guidance: Use `indexOf(delimiter)` + `slice()`, not `split(delimiter)`.

```
idx = str.indexOf(':');
user = idx == -1 ? str : str.slice(0, idx);
```

Rule 3: Validate Against Real Test Cases Early

Guidance: Locate and inspect the **exact failing test assertions** before implementing a fix.

E. Reflection and Improvement

Future efforts should avoid regex/text manipulation for code edits and prefer more precise tools. The final approach achieves idempotency ($f(f(x)) \equiv f(x)$), which is essential for robust normalization.

APPENDIX G
PROMPT DESIGN

we detail the prompts utilized in our experiments for both the vanilla agent and EVOREPAIR as follows. The system prompt is shared across all tasks, while the instance prompt is dynamically constructed for each remediation session based on specific vulnerability details. Additionally, the experience prompt is unique to EVOREPAIR.

Prompts used by EVOREPAIR

System Prompt
You are a helpful assistant that is equipped with tools to solve programming tasks.
Your response must contain exactly ONE tool call.
Include a THOUGHT section in your response where you explain your reasoning process.

Instance Prompt
<cve_description>
The vulnerability has the following CVE description:
{cve_description}
</cve_description>

<cwe_info>
The vulnerability has the following CWE information:
{cwe_info}
</cwe_info>

{% if experience_context %}
<historical_experiences>
The following are relevant experiences from similar vulnerability fixes .
These experiences are extracted from successful past fixes. Use them as reference to understand common patterns, but adapt the solutions to the current context rather than copying them directly:
{experience_context}
</historical_experiences>
{% endif %}

<vul_location>
The vulnerability is located in the following code lines, do not modify other code lines:
{vul_location}
</vul_location>

<instructions>

TASK INSTRUCTIONS OVERVIEW
You're a software engineer interacting continuously with a computer by calling tools. You'll be helping implement necessary changes to fix the vulnerability. Your task is specifically to make changes to non-test files in the current directory in order to fix the vulnerability described above in a way that is general and consistent with the codebase. To enhance your vulnerability remediation capabilities and experience, you should maintain a dedicated documentation for each vulnerability. This documentation should record details such as the vulnerability description, root cause analysis, remediation process, and outcomes. You can refer to previously documented records during the remediation process and continuously update and refine them for future reference.

IMPORTANT: This is an interactive process where you will think and call ONE tool, see its result, then think and call your next tool.

For each response:
1. Include a THOUGHT section explaining your reasoning and what you're trying to accomplish

2. Provide exactly ONE tool to call

IMPORTANT BOUNDARIES

- MODIFY: Regular source code files
- DO NOT MODIFY: Tests, configuration files (pyproject.toml, setup.cfg, etc.)

AVAILABLE SCRIPTS

The following four scripts are available to help you fix and verify the vulnerability:
bash fix-run.sh: Run fix-run.sh to execute the PoC and verify that the patched vulnerability is no longer exploitable. To run fix-run.sh, you must first generate fix.patch in the same directory.
bash vul-run.sh: Run vul-run.sh to execute the PoC and verify that the original (unpatched) vulnerability is exploitable.
bash unit_test.sh: (if present) Run unit_test.sh to execute unit tests and validate functional correctness.
bash prepare.sh: Resets all changes in the repository. Run this script before each evaluation.

RECOMMENDED WORKFLOW

1. Understand the vulnerability from the CVE description and CWE information
2. Check the latest documentation of the vulnerability if available
3. Verify the existence of the vulnerability by running vul-run.sh
4. Locate vulnerability in the source code if needed
5. Edit source code to fix the vulnerability
6. Use git diff to review your changes, generate fix.patch, and use cat to view its content
7. Verify fix.patch by running prepare.sh and fix-run.sh, if test error occurs, you can check test.patch for reference
8. If fix.patch passes verification, update documentation, submit fix.patch and finish workflow
Each response should include:
1. A THOUGHT section where you explain your reasoning and plan
2. A single tool call

CRITICAL REQUIREMENTS:

- Your response SHOULD include a THOUGHT section explaining your reasoning
- Your response MUST include EXACTLY ONE tool call

ENVIRONMENT DETAILS

- You have a full Linux shell environment
- Always use non-interactive flags (-y, -f) for commands
- Avoid interactive commands like vi, nano, or any that require user input
- If a command isn't available, you can install it
- Source code dir is {source_dir}, it is a git repository; you can use git commands to inspect changes
- The test.patch file documents the new tests that should pass after the vulnerability is fixed.

USEFUL COMMAND EXAMPLES

Create a new file
cat << 'EOF' > newfile.py
import numpy as np
hello = "world"
print(hello)
EOF

Edit files with sed

a. Replace all occurrences
sed -i 's/old_string/new_string/g' filename.py
b. Replace only first occurrence
sed -i 's/old_string/new_string/' filename.py
c. Replace first occurrence on line 1
sed -i '1s/old_string/new_string/' filename.py
d. Replace all occurrences in lines 1-10
sed -i '1,10s/old_string/new_string/g' filename.py

View file content
nl -ba filename.py — sed -n '10,20p'

Generate diff of changes
cd {source_dir} && git diff – . ':(exclude)*test*' ':(exclude)*Test*' > ../fix.patch

Validate the patch
prepare.sh is used to reset all changes, run it before validation.
please do not run unit tests for verification by yourself. Use fix-run.sh instead, because test.patch has not been included.
bash prepare.sh && bash fix-run.sh

Any other command you want to run
anything
</instructions>

APPENDIX H
EARLY STOP

To balance the performance and cost of EVOREPAIR, we have integrated an early-stopping strategy into our original methodology. The core rationale behind this strategy is that when the remediation cost of a single turn for EVOREPAIR outweighs the marginal gain from new fixes beyond a certain threshold, we deem the repair process no longer cost-effective and thus terminate the entire workflow prematurely. Specifically, we define the turn-level yield rate α (detailed in Equation 1) as the ratio between the marginal increase in repair rate and the marginal increase in cost. When α falls below a certain threshold t, EVOREPAIR triggers an early-stopping strategy. To facilitate the determination of the optimal threshold for α, Table V provides a detailed breakdown of EVOREPAIR's turn-level computational costs and incremental fixes, based on which the corresponding values of α are calculated. As shown, α exhibits a generally downward trend as the number of turns increases. By adjusting the threshold t, EVOREPAIR can achieve an optimal balance between performance and cost-effectiveness across different models. Based on the empirical data in the table, we recommend setting the threshold t within the range of 0.15 to 0.25.

$$\alpha = \frac{\beta}{\gamma} \tag{1}$$

Here, β and γ denote the growth rate of repair performance and the growth rate of repair cost, respectively.

TABLE V: Breakdown of new fixes and additional costs per turn on PATCHEVAL, used as a criterion for the early-stopping strategy.

Model	Turn	CVEs	New Fixes	Cumulative Fixes	New Cost ($)	Cumulative Cost ($)	β	γ	α
DeepSeek-v3.1	1	230	54	54	75.8867	75.8867	∞	∞	∞
	2	176	28	82	64.7944	140.6811	0.5185	0.8538	0.6073
	3	148	9	91	59.5065	200.1876	0.1098	0.423	0.2595
	4	139	13	104	51.6457	251.8333	0.1429	0.258	0.5537
	5	126	18	122	47.594	299.4273	0.1731	0.189	0.9158
	6	108	11	133	41.7326	341.1599	0.0902	0.1394	0.6469
	7	97	8	141	37.4431	378.603	0.0602	0.1098	0.5481
	8	89	5	146	36.1995	414.8025	0.0355	0.0956	0.3709
	9	84	3	149	30.5494	445.352	0.0205	0.0736	0.279
	10	81	1	150	29.284	474.6359	0.0067	0.0658	0.1021
	11	80	4	154	32.6795	507.3154	0.0267	0.0689	0.3873
	12	76	4	158	30.8755	538.191	0.026	0.0609	0.4268
	13	72	2	160	26.253	564.444	0.0127	0.0488	0.2595
	14	70	3	163	30.2974	594.7414	0.0187	0.0537	0.3493
	15	-	-	-	-	-	-	-	-
Qwen3.5-Plus	1	230	131	131	137.3843	137.3843	∞	∞	∞
	2	99	53	184	68.6887	206.073	0.4046	0.5	0.8092
	3	46	15	199	40.2869	246.36	0.0815	0.1955	0.417
	4	31	6	205	28.0739	274.4339	0.0302	0.114	0.2646
	5	25	2	207	20.4882	294.9221	0.0098	0.0747	0.1307
	6	23	2	209	22.8841	317.8062	0.0097	0.0776	0.1245
	7	21	3	212	17.7669	335.5731	0.0144	0.0559	0.2568
	8	18	2	214	18.2184	353.7915	0.0094	0.0543	0.1738
	9	16	4	218	11.7496	365.5411	0.0187	0.0332	0.5628
	10	12	1	219	12.4055	377.9466	0.0046	0.0339	0.1352
	11	11	1	220	10.1384	388.0851	0.0046	0.0268	0.1702
	12	10	0	220	10.2706	398.3557	0	0.0265	0
	13	-	-	-	-	-	-	-	-
	14	-	-	-	-	-	-	-	-
	15	-	-	-	-	-	-	-	-
Devstral-123B	1	230	124	124	104.8407	104.8407	∞	∞	∞
	2	106	40	164	53.7326	158.5732	0.3226	0.5125	0.6294
	3	66	15	179	36.4763	195.0495	0.0915	0.23	0.3976
	4	51	13	192	26.5779	221.6274	0.0726	0.1363	0.533
	5	38	3	195	24.9846	246.612	0.0156	0.1127	0.1386
	6	35	6	201	19.3673	265.9793	0.0308	0.0785	0.3918
	7	29	4	205	18.6482	284.6275	0.0199	0.0701	0.2838
	8	25	2	207	15.5734	300.2009	0.0098	0.0547	0.1783
	9	23	1	208	14.7502	314.9511	0.0048	0.0491	0.0983
	10	22	2	210	11.872	326.8231	0.0096	0.0377	0.2551
	11	20	1	211	8.8812	335.7044	0.0048	0.0272	0.1752
	12	19	2	213	8.6617	344.3661	0.0095	0.0258	0.3674
	13	17	2	215	10.5507	354.9168	0.0094	0.0306	0.3065
	14	15	1	216	8.9027	363.8195	0.0047	0.0251	0.1854
	15	-	-	-	-	-	-	-	-
Devstral-24B	1	230	113	113	28.4529	28.4529	∞	∞	∞
	2	117	49	162	18.1357	46.5886	0.4336	0.6374	0.6803
	3	68	17	179	11.6543	58.243	0.1049	0.2502	0.4195
	4	51	12	191	9.8368	68.0797	0.067	0.1689	0.3969
	5	39	4	195	7.7828	75.8625	0.0209	0.1143	0.1832
	6	35	4	199	6.9982	82.8607	0.0205	0.0922	0.2224
	7	31	3	202	6.1207	88.9814	0.0151	0.0739	0.2041
	8	28	5	207	5.8086	94.79	0.0248	0.0653	0.3792
	9	23	6	213	3.9927	98.7828	0.029	0.0421	0.6881
	10	17	1	214	3.4361	102.2188	0.0047	0.0348	0.135
	11	16	2	216	2.7336	104.9524	0.0093	0.0267	0.3495
	12	14	1	217	2.6001	107.5525	0.0046	0.0248	0.1869
	13	13	1	218	2.4921	110.0446	0.0046	0.0232	0.1989
	14	12	0	218	2.4103	112.4548	0	0.0219	0
	15	-	-	-	-	-	-	-	-
GPT-5-mini	1	230	105	105	41.1859	41.1859	∞	∞	∞
	2	125	49	154	28.9068	70.0927	0.4666	0.7019	0.6648
	3	76	22	176	17.4825	87.5752	0.1429	0.2494	0.5728
	4	54	12	188	12.1541	99.7293	0.0682	0.1388	0.4913
	5	42	7	195	12.2516	111.9809	0.0372	0.1228	0.3031
	6	35	3	198	8.5106	120.4915	0.0154	0.076	0.2024
	7	32	4	202	12.4333	132.9248	0.0202	0.1032	0.1958
	8	28	2	204	14.608	147.5327	0.0099	0.1099	0.0901
	9	26	2	206	9.4343	156.967	0.0098	0.0639	0.1533
	10	24	4	210	9.369	166.336	0.0194	0.0597	0.3253
	11	20	1	211	7.0953	173.4313	0.0048	0.0427	0.1116
	12	19	1	212	9.1991	182.6304	0.0047	0.053	0.0894
	13	18	2	214	4.9421	187.5725	0.0094	0.0271	0.3486
	14	16	1	215	6.8364	194.4089	0.0047	0.0364	0.1282
	15	-	-	-	-	-	-	-	-